

Phase 2 — Unified Enterprise Application Development

Role

Continue acting as a **Principal Software Architect & Senior Google Apps Script Engineer (15+ years of industry experience)** with expertise in:

- Google Apps Script
- Google Workspace Automation
- Enterprise Web Applications
- JavaScript (ES6+)
- HTML/CSS
- UI/UX Design
- Software Architecture
- Authentication & Authorization
- Session Management
- Google Sheets as a Database
- System Design
- Performance Optimization
- Secure Application Development

You are expected to design and implement this application as though it will be deployed in a production enterprise environment.

Background

You have already completed a comprehensive analysis of the following projects:

- New Program Addition
- Attendance
- Shiksha
- Unified

From your analysis, you should now have a complete understanding of:

- Application architecture
- Business workflows
- Data flow
- Google Sheets integration
- UI structure
- Existing limitations
- Cross-project dependencies

The existing **Unified** project already implements approximately **60%** of the functionality required for the final system.

It should therefore be treated as the **primary architectural reference**, not necessarily as production-ready code.

Reuse architectural concepts where appropriate, but redesign and improve areas that are incomplete, inconsistent, or poorly implemented.

Objective

Develop the next-generation version of the application.

Create a **new project folder** named:

unified_28_06_2026

Do **not** modify the existing **Unified** project.

The new folder should become the production-ready implementation that consolidates and improves the functionality of all previous projects.

Development Principles

While developing:

- Follow clean architecture principles.
- Keep business logic modular.
- Avoid duplicate code.
- Separate frontend and backend responsibilities.
- Design reusable utility functions.
- Keep HTML, CSS, and JavaScript modular.
- Maintain readable naming conventions.
- Build scalable Apps Script services.
- Design for future feature additions.

Every implementation decision should prioritize:

- Maintainability
- Scalability
- Performance
- Security

- Readability

Authentication System

The authentication system must use the existing **Credentials Sheet**.

The credentials sheet contains at least:

Name	Password	Role
------	----------	------

The **Role** column determines application access.

Currently there are two roles:

- Owner
- Admin

Future roles may be added, so the implementation should remain extensible.

Login Flow

After successful login:

1. Validate username.
2. Validate password.
3. Read user role.
4. Create a user session.
5. Store all required session information.
6. Redirect the user to the main application.

The application should **not repeatedly authenticate the same logged-in user during the active session**.

Owner Experience

If the logged-in user has the role:

Owner

They should experience the standard application workflow exactly as designed.

No unnecessary admin options should be displayed.

Admin Experience

If the logged-in user has the role:

Admin

The interface must include an additional option such as:

- Admin Console
- Admin Dashboard
- Administration

The label may be chosen based on the overall UI consistency.

Admin Console Requirement

This is one of the highest-priority requirements.

Current Problem

In the existing **Unified** project:

- Admin logs in.
- Admin clicks **Admin Console**.
- The application asks the admin to log in again.

This is **incorrect behavior**.

Required Behavior

After an Admin successfully logs in:

- Their authenticated session should remain active.
- Clicking **Admin Console** should directly open the admin interface.
- No second login should occur.
- No credential prompt should appear.
- No authentication screen should reload.

The system should recognize that:

The user has already been authenticated.

Navigation between the user interface and the admin interface should occur within the same authenticated session.

Session Management

Implement proper session management.

The session should securely maintain:

- Username
- User role
- Login status
- Authorization state
- Any required user metadata

Session persistence should be sufficient to support seamless navigation throughout the application without unnecessary re-authentication, while still allowing secure logout and session expiration when appropriate.

Authorization

Authentication and authorization should remain separate concerns.

Authentication verifies identity.

Authorization determines access.

Every protected action should verify the user's role before allowing access.

For example:

- Admin pages require Admin authorization.
 - Owner-only features remain restricted where applicable.
 - Future roles should be easy to integrate with minimal changes.
-

UI / UX Expectations

Maintain a modern, professional, and consistent interface.

Requirements include:

- Clean dashboard layout
- Responsive design
- Consistent navigation
- Clear visual hierarchy
- Reusable components
- Professional styling
- Smooth transitions where appropriate
- Minimal unnecessary page reloads

Navigation between user and admin sections should feel seamless.

Code Organization

Organize the project into logical modules.

Examples include:

- Authentication
- Session Management
- Authorization
- Google Sheets Services
- User Services
- Admin Services
- Utilities
- Shared Components
- Frontend Scripts
- UI Components

Avoid creating monolithic files.

Compatibility

The application should remain compatible with the existing Google Sheets structure unless a justified architectural improvement is required.

Reuse existing data wherever possible instead of introducing unnecessary duplication.

Implementation Strategy

Before writing new code:

1. Review the corresponding implementation in the existing **Unified** project.
 2. Determine why it works or fails.
 3. Improve the architecture where necessary.
 4. Preserve successful patterns.
 5. Replace weak implementations with scalable solutions.
-

Expected Deliverables

The final implementation should include:

- A new project folder named `unified_28_06_2026`
 - Clean project architecture
 - Robust authentication
 - Persistent session management
 - Role-based authorization
 - Seamless Admin Console navigation
 - Improved modular codebase
 - Production-quality UI
 - Reusable utilities
 - Clear separation of concerns
 - Extensible design for future roles and features
-

Important Rules

- Do **not** modify the original **Unified** project.
- Use it only as a functional and architectural reference.
- Do **not** copy flawed implementations.
- Improve architecture wherever necessary.
- Keep every new module production-ready.
- Avoid quick fixes or hard-coded logic.
- Prioritize long-term maintainability over short-term convenience.

The end goal is to produce a stable, scalable, enterprise-grade Google Apps Script application that unifies all existing projects into a single, maintainable codebase with a secure, role-based authentication and authorization system.